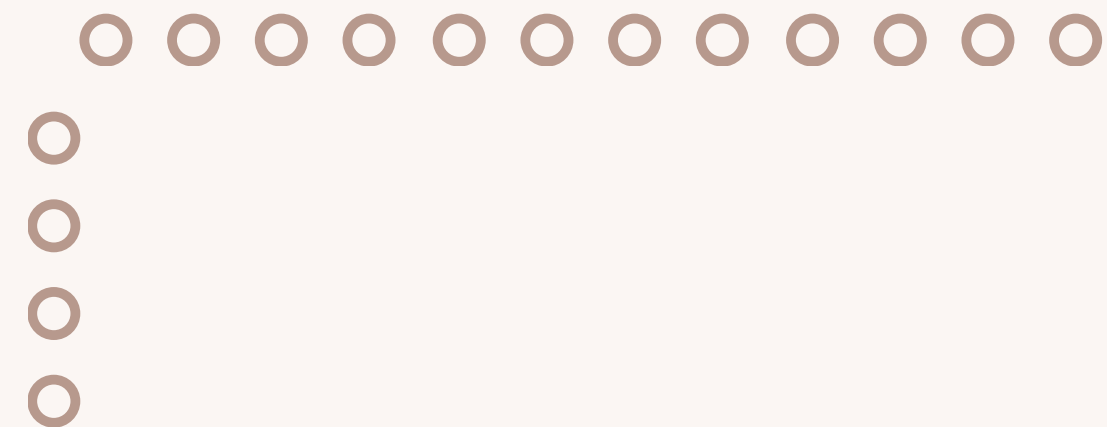
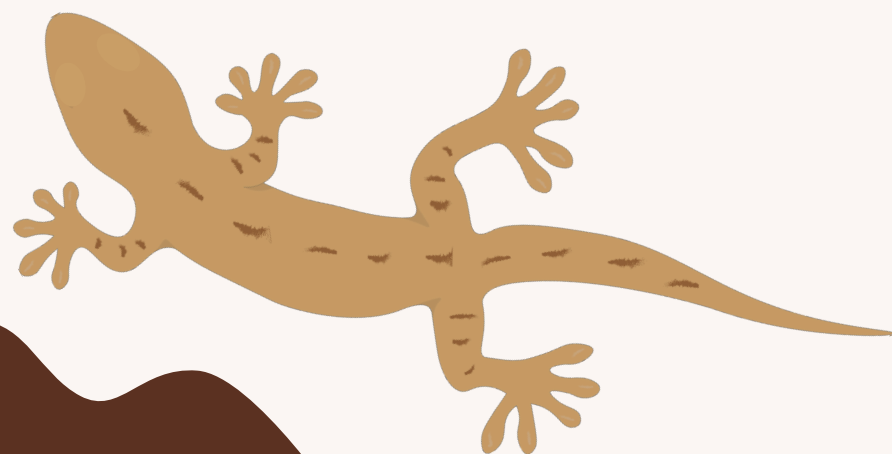
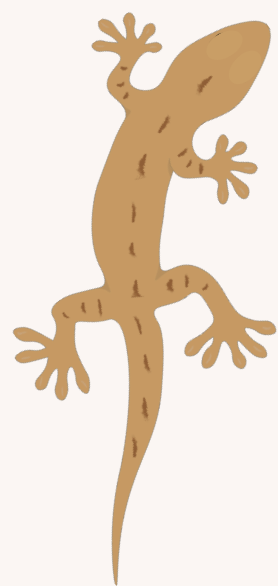
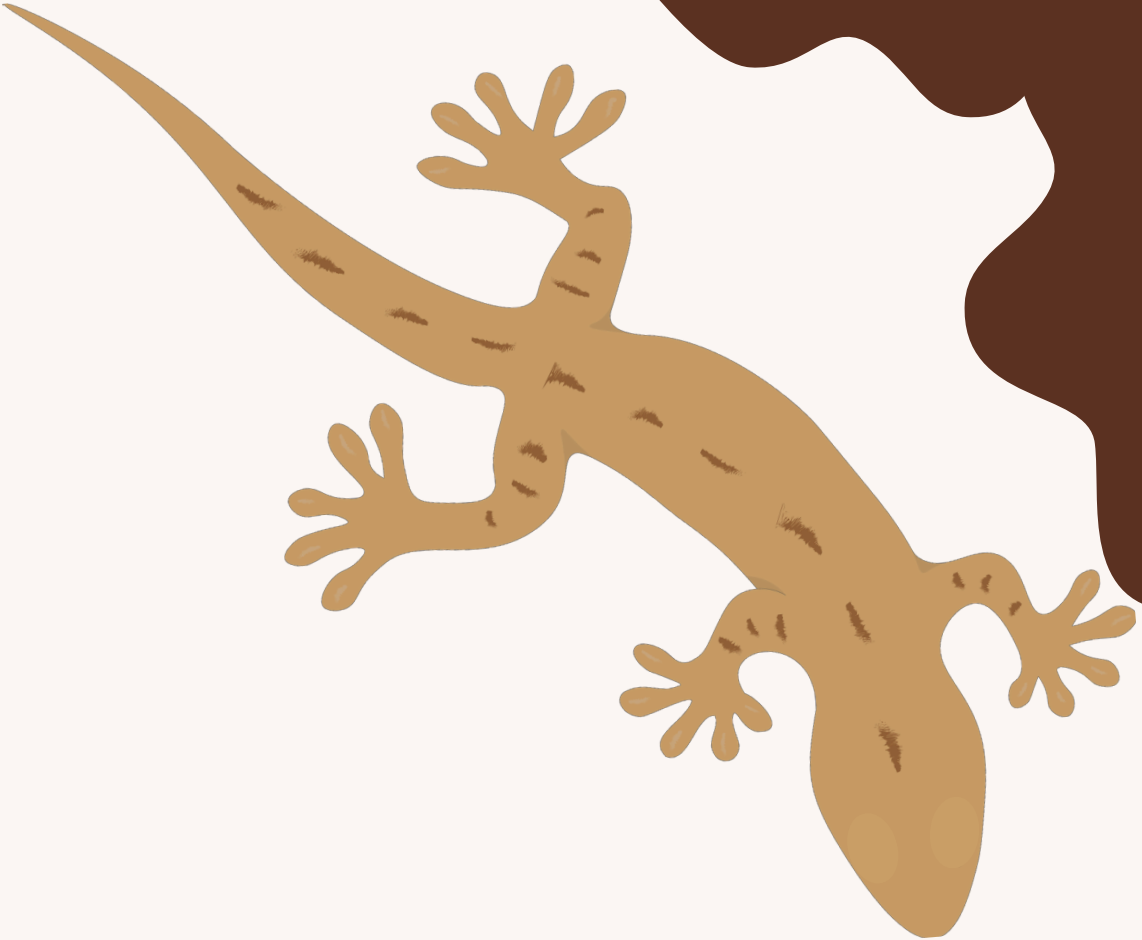
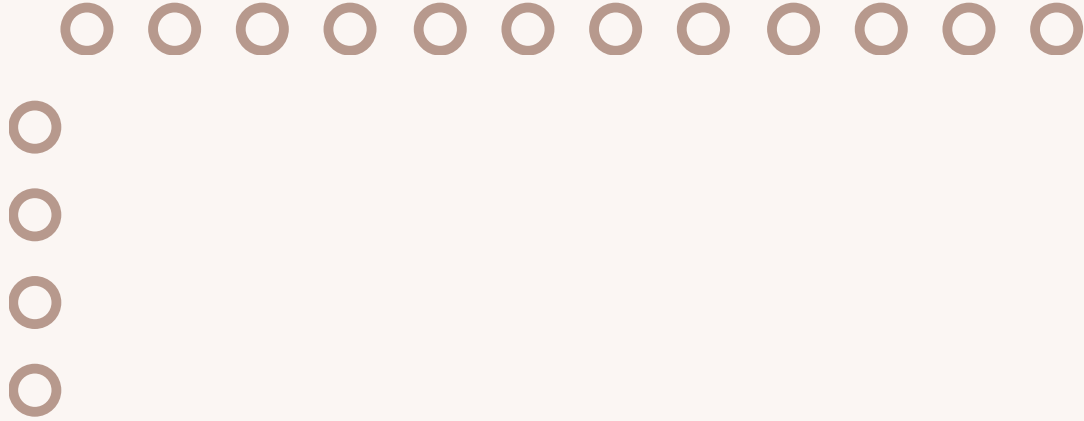




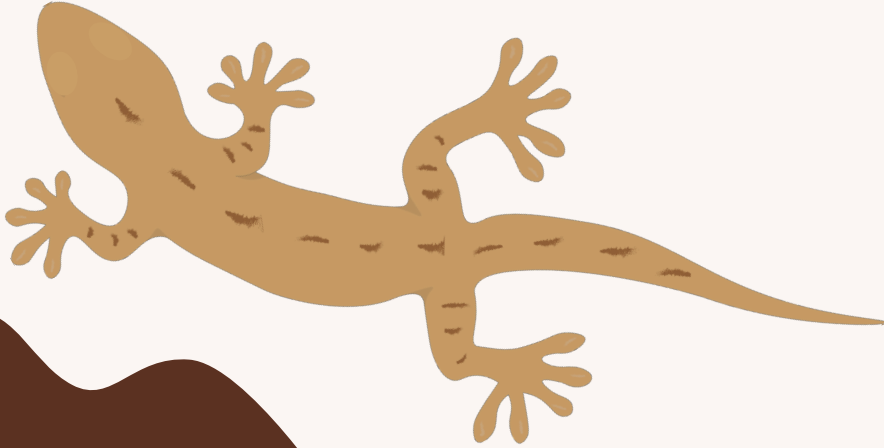
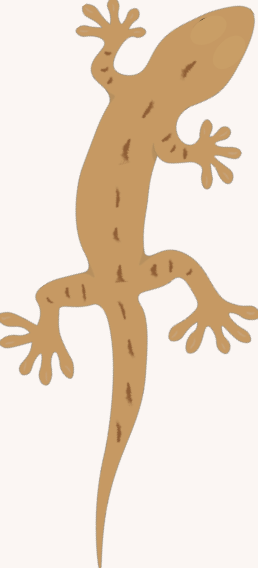
Presented by Strawhat pirates





GREPO

Grepo is a developer tool that helps users understand unfamiliar codebases by visualizing their structure, tracing source-linked relationships, and surfacing code worth investigating.





Why Grepo?

- **Source-linked insights:** Every finding points back to the files and lines that support it.
- **Transparent analysis:** Unresolved relationships are clearly identified as limitations rather than presented as facts.
- **Built for verification:** Developers can inspect the underlying code and validate findings in the same workspace.

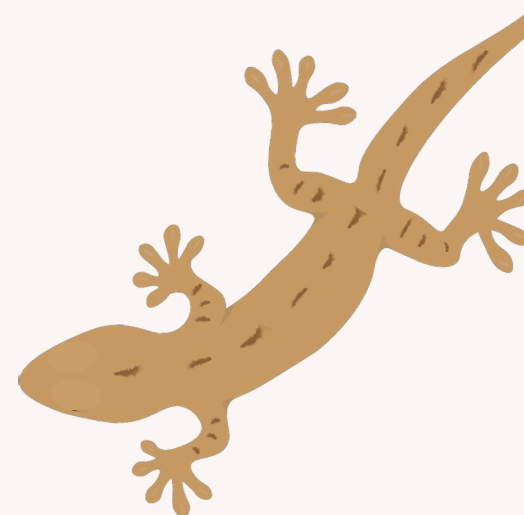
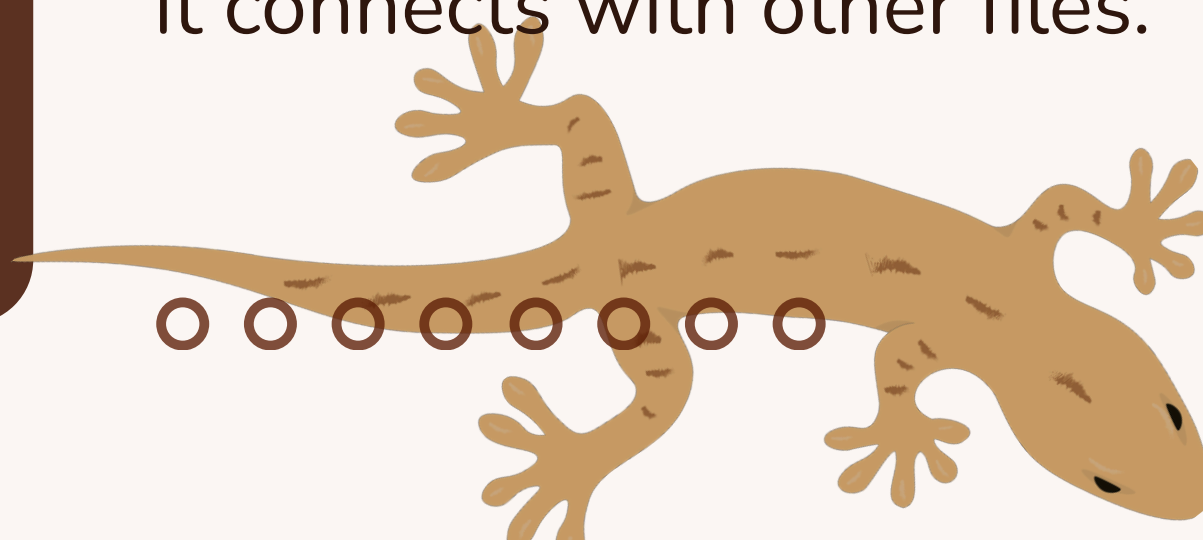


- Finding the right files and functions takes time.
- Dependencies and change impact are hard to trace.
- Documentation may be incomplete or out of date.
- Existing reusable code can be easy to miss.

Problems

Large codebases can be difficult to navigate, especially when you're new to a project.

Before making a change, developers often have to first figure out where the relevant code is and how it connects with other files.



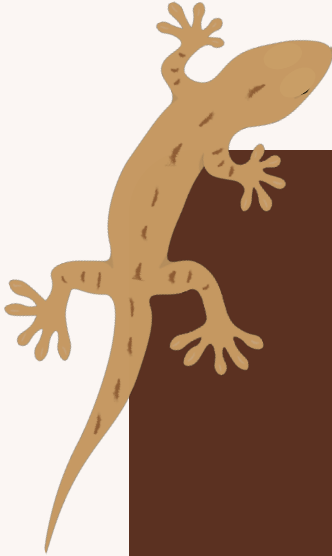


Solution

Grepo turns a ZIP or public GitHub repository into an interactive workspace.

Explore the project structure, inspect source-linked summaries and relationships, and review potential reuse, duplicate, and unused-code findings.

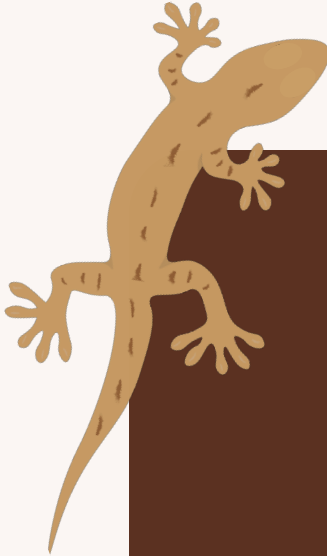




Features

- **Project Structure:** Searchable file tree and interactive architecture map.
- **Generate Summaries:** Source-cited summaries you can verify against the code.
- **Dependencies:** Explore discovered relationships and understand the potential impact of code changes.





Features

- **Reusable Functions:** Find functions that appear to be called across files.
- **Duplicate and Unused Functions:** Find functions that are potentially useless.
- **Codebase AI Chat:** Ask questions about the repository with optional AI chat feature.



How it works

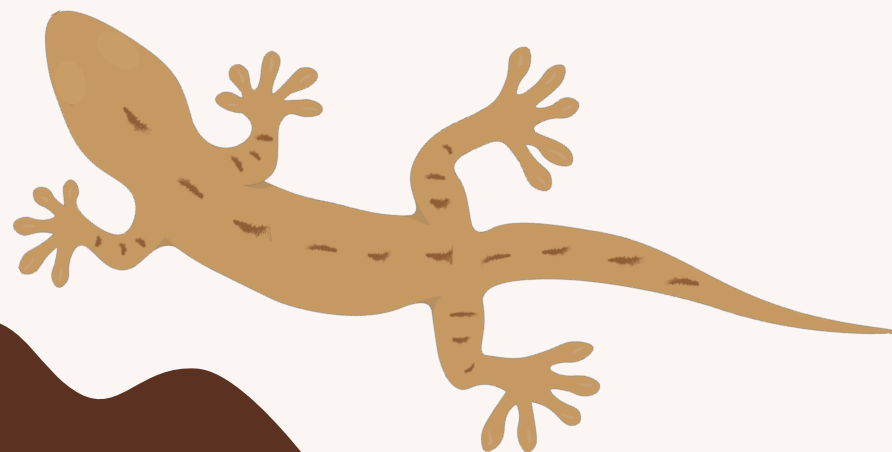
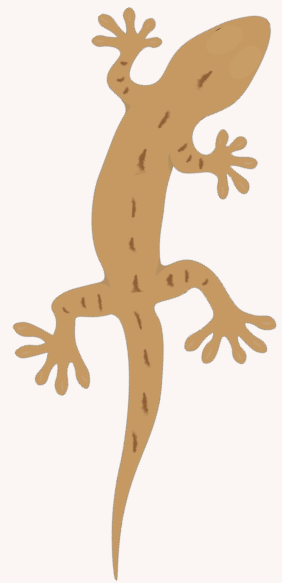
Codebase

Safe import into a snapshot

Static analysis

Source-linked graph and findings

Grepo workspace



System Architecture

React + TypeScript frontend

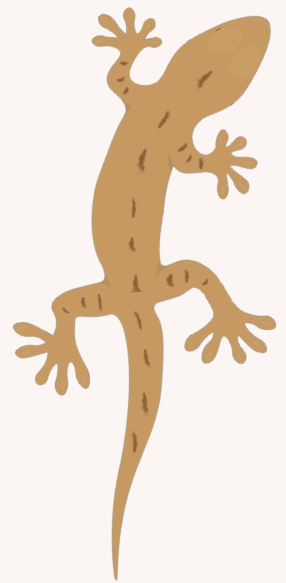
FastAPI backend

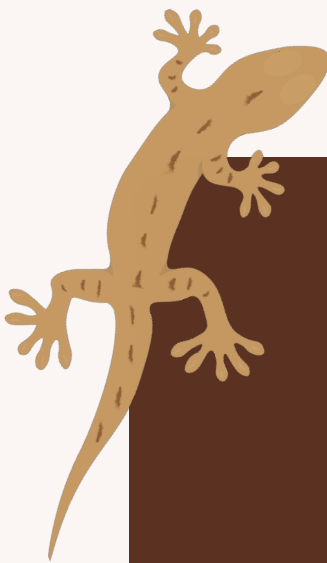
Import and snapshot services

Python AST/Tree-sitter analysis

Snapshot data, evidence, and graph

Interactive map rendered with Archify



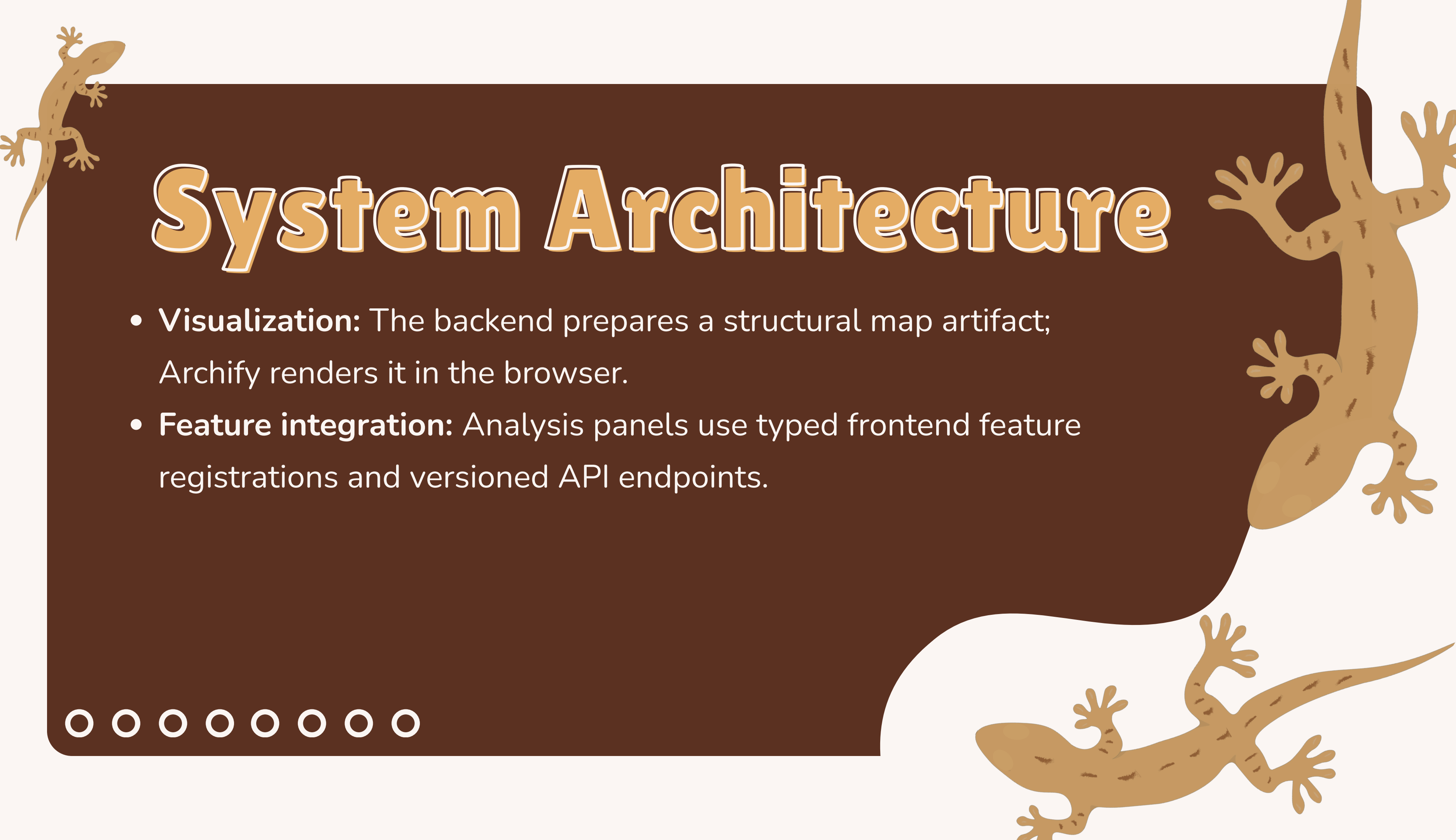


System Architecture



- **Frontend:** React and TypeScript provide the searchable tree, map, source viewer, and analysis panels.
- **Backend:** FastAPI handles repository imports, snapshot access, analysis requests, and source retrieval.
- **Analysis:** Python files use Python's AST; Tree-sitter extracts syntax facts across additional languages. Dependency resolution is implemented for Python and JavaScript/TypeScript.





System Architecture

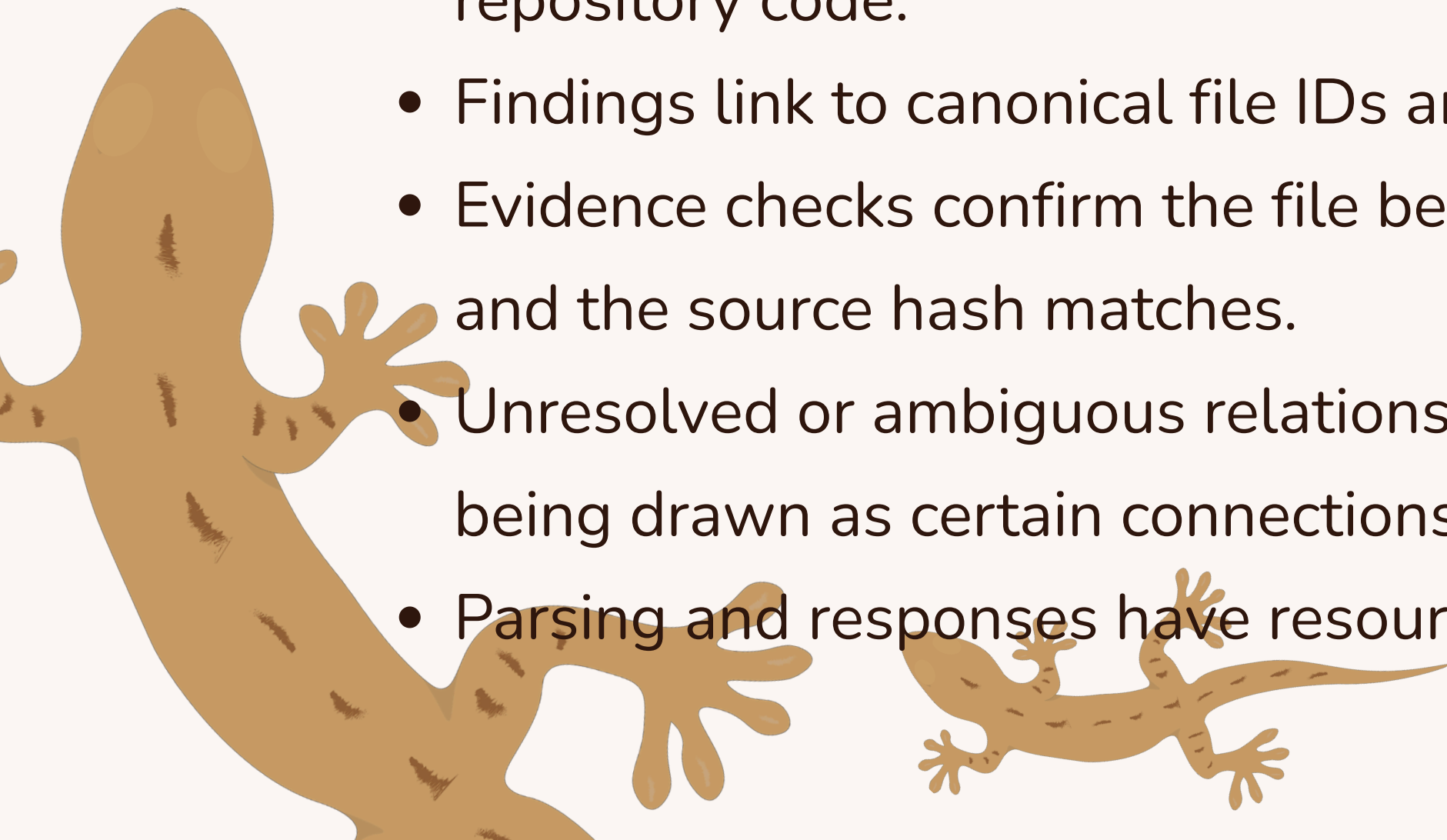
- **Visualization:** The backend prepares a structural map artifact; Archify renders it in the browser.
- **Feature integration:** Analysis panels use typed frontend feature registrations and versioned API endpoints.





Evidence and Safety

- Imports become immutable, snapshot-scoped workspaces.
- Analysis reads extracted syntax and source; it does not execute imported repository code.
- Findings link to canonical file IDs and line ranges.
- Evidence checks confirm the file belongs to the snapshot, the cited range is valid, and the source hash matches.
- Unresolved or ambiguous relationships are reported as limitations instead of being drawn as certain connections.
- Parsing and responses have resource and size bounds.



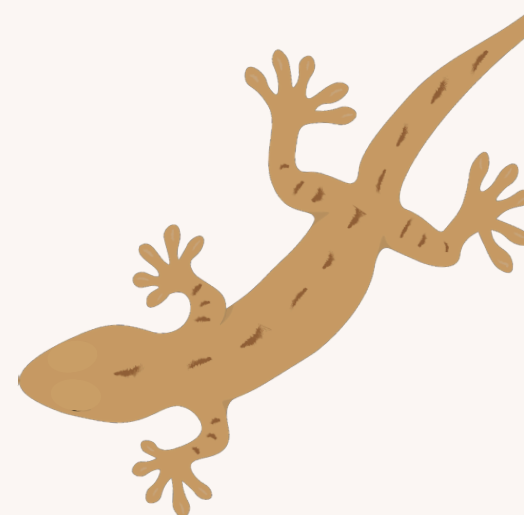
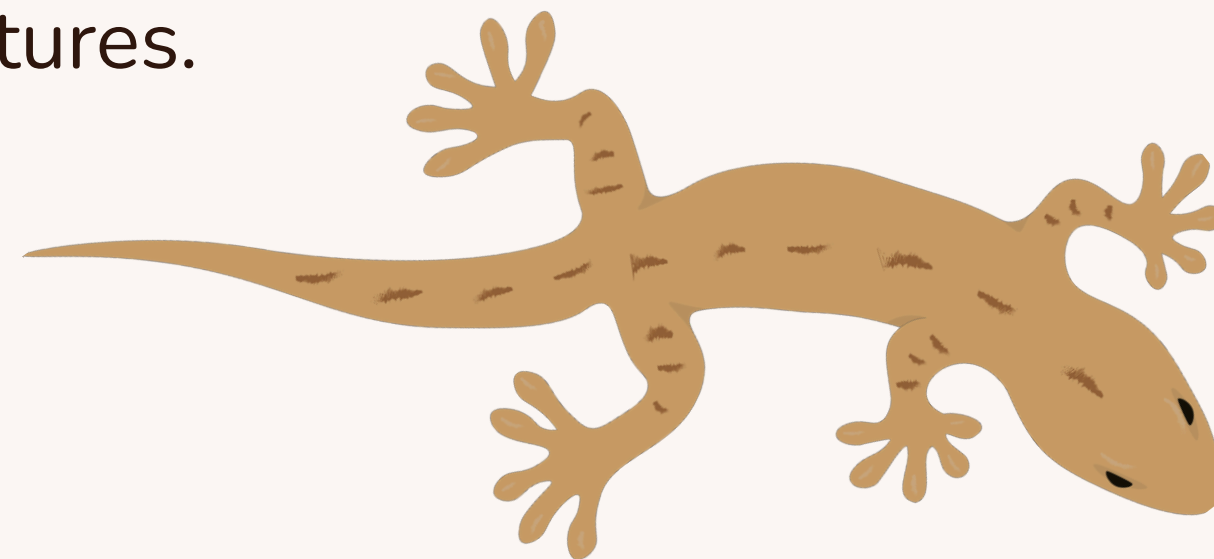
- **Summaries:** validated the cited-summary approach, then implemented a deterministic summaries engine with evidence verification.
- **Dependencies:** built dependency analysis and change-impact behavior, then added guards for defects found through testing.
- **AI Chatbot:** wired the Ask feature into the interface and fixed response-rendering issues.
- **Workspace:** supported the initial structure workspace implementation and repository setup.



IBM BOB

Development Workflow

We used IBM Bob throughout development to investigate, implement, test, and refine Grepo features.



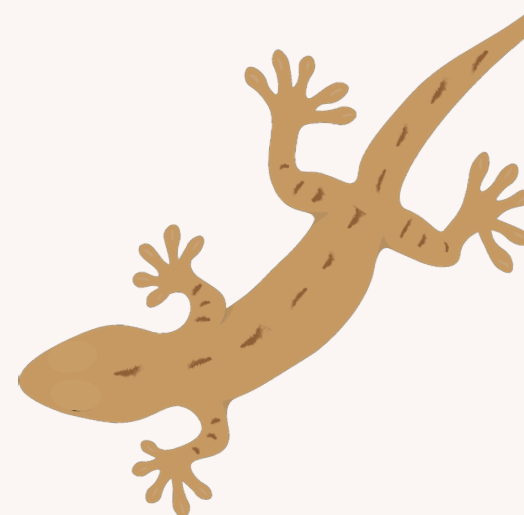
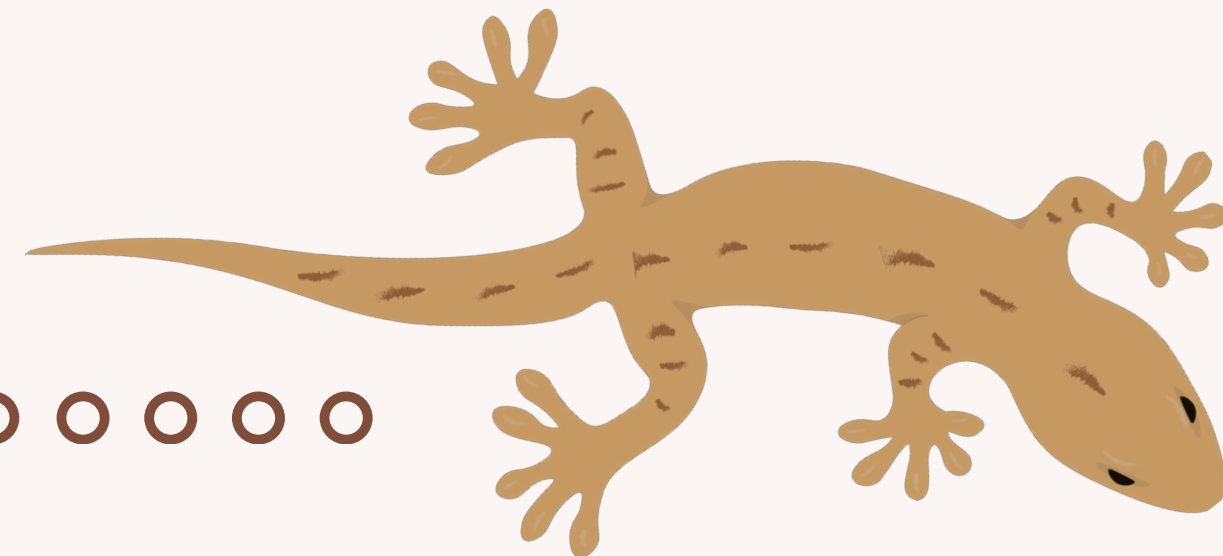
- Arjun: repository foundation and core workspace setup.
- Jae: cited summaries and dependency analysis.
- Sajid: grounded repository chat with source-linked answers.
- Corrections for root selection, dependency targets, Ask wiring, greetings, and answer rendering.
- Evidence is preserved by task.
- Archived task IDs, transcripts, session summaries, and available consumption panels.



IBM BOB

Session Evidence

Bob session evidence is organized in numbered task folders under bob_sessions. Each folder preserves the task's original ID and available summary, transcript, and screenshots.



IBM BOB

Reflection by Hero

Audit the starting point, then build on shared foundations.

1. Audit the template

Find edge cases, integration gaps and duplicated logic before building.

2. Build from a plan

Extend shared parsing and reuse analysis in defined steps.

3. Use shared standards

Align language maps, enums, API wrappers and source ranges.

4. Scope the routines

Request frontend, backend and test work; inspect the reported checks.

Audit: edge cases identified before implementation.

Issue	Fix Complexity
<code>CallSite</code> used before definition – <code>NameError</code> at import	1 line – move class up
Duplicate language extension map	~10 lines – delete + redirect
<code>SlotStatus</code> and <code>CapabilityLevel</code> are plain <code>str</code> subclasses, not enums	2 lines each – add <code>Enum</code> base
<code>SLOT_UNUSED.integration_path</code> points to <code>duplicate_detection</code>	1 line – fix string
<code>OverviewPage</code> is a pass-through to <code>MapPage</code>	Delete 1 file, 1 line in <code>App.tsx</code>

Applied fixes: source ranges and request wrappers.

```
Removed start === lineStart ? lineEnd : undefined guard – lineEnd is now always forwarded, matching how the reset effect on line 8 works
```

```
Rewrote to use ApiError from v1/api and a shared legacyRequest<T>() helper – eliminates the duplicate error-extraction boilerplate
```



Session screenshots from Hero. Cropped excerpts. Full evidence.



IBM BOB

Hero in practice

Recorded implementation and verification from the development session.

Parsing foundations

Python AST call-site extraction;
initial JS/TS and Java regex
analyzers.

Reuse, end to end

Definition and caller indexes,
a reuse endpoint, and a UI with
source navigation.

Reported checks

Session summary reports analyzer,
cross-file, threshold and UI-state
checks.

Implementation: Python AST and initial regex analyzers.

```
Extended to walk ast.Call nodes → populates call_sites on every parsed File

New – regex-based JS/TS analyzer, covers .js .jsx .ts .tsx .mjs .cjs. Extracts functions, call_sites, imports

New – regex-based Java analyzer for .java. Extracts methods (skips constructors), call sites, import statements
```

Reported tests: extraction, reuse rules and panel states.

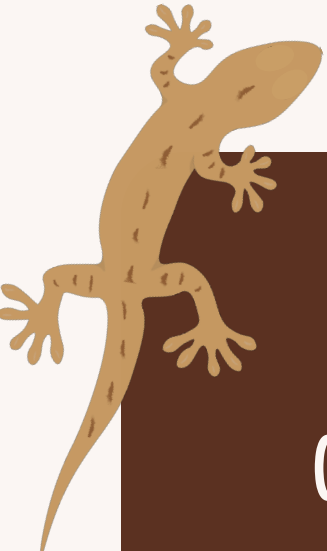
Coverage
Python call_sites, JS function/call/arrow/TS, Java method/call/import
Cross-file detection, same-file exclusion, min_callers threshold, missing syntax graceful empty
Empty state, group rendering, openSource callback, loading state

Subagent routines: requested backend, frontend and test roles.

```
Update Todo List (failed)
[-] Spawn backend implementation subagent (1A-3A)
[-] Spawn frontend implementation subagent (4A-5D)
[-] Spawn tests subagent (6A-6C)
```

Session screenshots from Hero. Cropped excerpts. Full evidence.



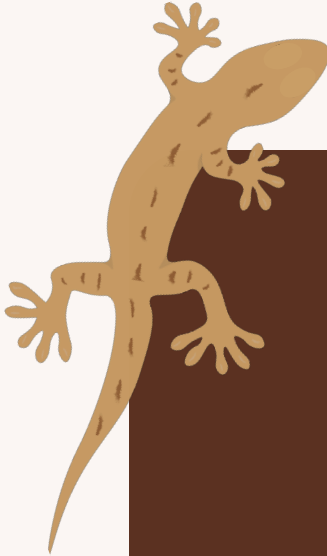


Impact

Grepo aims to help developers:

- Clearly navigate through unfamiliar codebases faster.
- Trace relationships before changing code.
- Find existing code that may be reusable



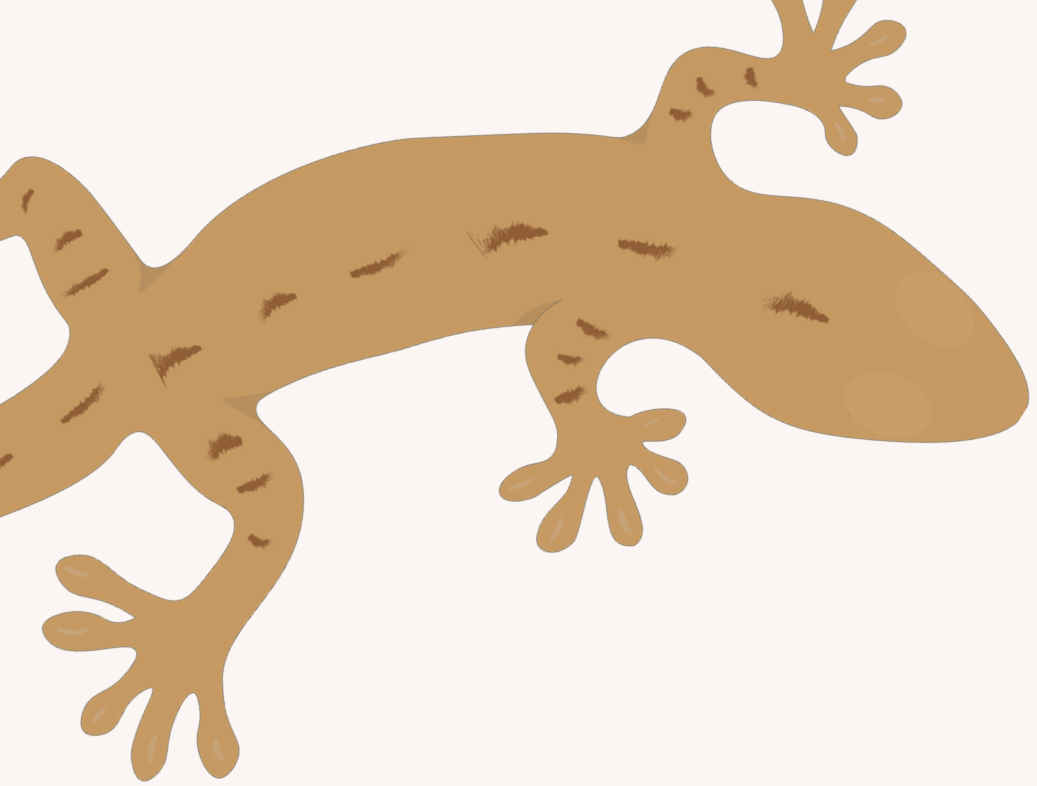


Future

Grepo aims to improve and add on:

- Broader language support
- Deeper semantic analysis
- Stronger summaries and Q&A





THANK YOU

